

15B17CI371 – Data Structures Lab
ODD2026
Week 1, Practice Lab [CO: C270.1]

Instructions:

1. **All students must save their programs using the following naming convention:** (Enroll. No_W1) with docx or pdf formats. For each question, mention the question, its solution C++ program, and its execution output.
2. **Students are advised to write code for all questions** to ensure thorough practice of all related topics.

Code of Conduct:

1. **Avoid using mobile phones, earphones, or watching YouTube videos during lab hours.** Defaulters will receive zero marks in evaluations.
2. **AI Tool Usage Policy:** Zero marks will be awarded for the sole or extensive use of AI tools.

Do not's	Dos
Do not solely/extensively use AI tools to draft your abstract, synopsis, report, etc.	You may use AI tools for spelling or grammar checking as you write.
Do not use AI tool to summarize or complete your project code	You may use AI to search relevant materials, and do programming by yourself.
Do not use AI tool to complete, analyse or rectify errors in your project code.	You may ask your mentor to help with your code if you get stuck.

Concepts: Stacks & Queues

Practice from Virtual Lab Portal. Link: <https://ds1-iiith.vlabs.ac.in/exp/stacks-queues/index.html>

Create an additional document with screenshots added and upload in your GCR

Practice Lab Questions:

Q1. Given a string 's' containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid. Assume that the string can contain parentheses only i.e., only '()[]{}' characters are allowed; and the maximum length of string can be, say, 20 characters. An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1: Input: s = "()"	Example 2: Input: s = "([)]"	Example 3: Input: s = "([)]"
Output: true	Output: true	Output: false

Q2. Given an array of numbers, input one number from this array and find if it's next-greater-element exists to the right of this value in the array. If yes, give the position; else print "Not found". Use stacks to perform this operation.

Example 1:

Input: arr[] = {1,4,2,5,0,6,7}

Input: element= 4

Output: 2 (element 5 is the first next-greater-element which exists to the right)

of the given element in the array, and it is 2 positions far from 4).

Example 2:

Input: arr[] = {1,4,2,5,0,6,7}

Input: element= 2

Output: 1 (element 5 is the first next-greater-element which exists to the right of the given element in the array, and it is 1 position far from 2).

Example 3:

Input: arr[] = {10,4,2,5,0,6,7}

Input: element= 7

Output: "Not found" (There are no element to the right of 7 in this array)

Example 4:

Input: arr[] = {10,6,7,2,5,1,0,4}

Input: element= 7

Output: "Not found" (There are elements to the right of 7, but none of them are greater than 7).

Q3. Modify the above question to allow for circular search.

Example 1:

Input: arr[] = {1,4,2,5,0,6,7}

Input: element= 4

Output: 2 (element 5 is the first next-greater-element which exists to the right of the given element in the array, and it is 2 positions far from 4).

Example 2:

Input: arr[] = {1,4,2,5,0,6,7}

Input: element= 2

Output: 1 (element 5 is the first next-greater-element which exists to the right of the given element in the array, and it is 1 position far from 2).

Example 3:

Input: arr[] = {10,4,2,5,0,6,7}

Input: element= 7

Output: 1 (Since you are allowing circular search, once the array reaches the end, start from the beginning. There now exists 10 which is greater than 7 and it can be said to travel 1 hop after 7).

Example 4:

Input: arr[] = {10,6,7,2,5,1,0,4}

Input: element= 7

Output: 6 (Since you are allowing circular search, once the array reaches the end, start from the beginning. There now exists the value 10 which is greater than 7 and it can be said to travel 6 hops to reach the position where currently 10 resides).

Q4. You are given a string 's', find the first non-repeating character (a character that occurs only once) in it and return its index (position). If it does not exist, return -1. Implement using concepts of queues.

Example 1:

Input: s =
"thisisDSlab" Output:
Character: t
Index: 0

Example 2:

Input: s = "CodeForDSlabClass"
Output: Character: d
(Assumption - 'd' and 'D' are considered as different characters) Index:

Example 3:

Input: s = "The quick brown fox jumps over a lazy dog"
Output: Character: None
Index: -1

Q5. Write a program to convert a number from decimal notation to a number expressed in a number system whose base (or radix) is a number between 2 and 9. The conversion is performed by repetitious division by the base to which a number is being converted and then taking the remainders of division in the reverse order. For **example**, in converting to binary, the number 6 requires three such divisions: $6/2 = 3$ remainder 0, $3/2 = 1$ remainder 1, and finally, $1/2 = 0$ remainder 1. The remainders 0, 1, and 1 are put in the reverse order so that the binary equivalent of 6 is equal to 110. Use stacks/queues to implement the conversion of an integer of base 10 to binary

Q6. Write a program to convert (a) given postfix to prefix (b) given prefix to postfix (c) given infix to postfix and further evaluate it to obtain the computed value. Example input: $(4 + 9 * 6) - ((8 - 6) / 2 * 4) * 9 / 3$

Q7. Write a program to check for balancing symbols (parentheses forms) in the following languages: [], {}, ()

Q8. Write a program to compress a given text by removing whitespaces and replacing continuously repeated characters by character followed by no. of times it is repeated. Use a queue data structure.

Example:

Input: asdddfghjdffkj
Output: asd3fghjdf2kj

Q9. Write the definition of the function '**moveNthFront**' that takes as a parameter a positive integer, n. The function moves the nth element of the queue to the front. The order of the remaining elements remains unchanged.

Example

Input queue = {5, 11, 34, 67, 43, 55} and n = 3.
After a call to the function moveNthFront,

Output queue = {34, 5, 11, 67, 43, 55}.

Q10. Write a program that reads a line of text, changes each uppercase letter to lowercase, and places each letter both in a queue and onto a stack. The program should then verify whether the line of text is a palindrome (a set of letters or numbers that is the same whether read forward or backward).

Q11. Given string num representing a non-negative integer num, and an integer k, return the smallest possible integer after removing k digits from num.

Input: num = "1432219", k = 3

Output: "1219"

Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.

Q12. You are given a queue containing N integers and an integer K ($K \leq N$). Reverse only the first K elements of the queue while keeping the remaining elements in the same relative order. Implement the solution using both **Queue** and **Stack** data structures.

Example 1:

Input: Queue = {10, 20, 30, 40, 50}; K = 3

Output: Queue = {30, 20, 10, 40, 50}

Example 2:

Input: Queue = {5, 8, 2, 9, 1, 6}; K = 4

Output: Queue = {9, 2, 8, 5, 1, 6}

Q13. Write a C++ program to evaluate a given **Prefix Expression** using a **Stack**. The expression may contain single-digit operands and the operators: +, -, *, /, ^

Example 1:

Input Prefix Expression: -*+4235

Output: Result = 15

Example 2:

Input Prefix Expression: +9*26

Output: Result = 21